

Recipe Vibe App - Implementation Guide for Claude Opus 4.7

Purpose: Quick reference for building Recipe Vibe App locally on macOS using free tech stack.

Quick Summary

App Purpose: Users describe mood ("cozy comfort food") → Get recipe recommendations with AI explanations.

Tech Stack (All Free):

- Frontend: React + Vite + Tailwind + DaisyUI
- Backend: Python + FastAPI + Uvicorn
- Database: Supabase (PostgreSQL + pgvector)
- Embeddings: Sentence-Transformers (local, 384-dim)
- LLM: Ollama + Mistral 7B (local)
- RAG: LangChain
- Hosting: Vercel (frontend) + Railway (backend)

Data: 2.5M recipes (Food.com 230K + RecipeNLG 2.2M + Indian 10K)

Project Directory Structure

```
recipe-vibe-app/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   │   ├── MoodSelector.jsx  
│   │   │   ├── InventoryInput.jsx  
│   │   │   ├── RecipeCard.jsx  
│   │   │   ├── RecipeList.jsx  
│   │   │   └── LoadingState.jsx  
│   │   ├── pages/  
│   │   │   ├── Home.jsx  
│   │   │   └── RecipeDetail.jsx  
│   │   ├── services/  
│   │   │   └── api.js  
│   │   └── App.jsx
```

```
| | └─ main.jsx
| └─ package.json
| └─ vite.config.js
| └─ tailwind.config.js
|
└─ backend/
  └─ app/
    └─ main.py
    └─ routes/
      └─ recipes.py
    └─ services/
      └─ rag_pipeline.py
      └─ embeddings.py
      └─ llm_service.py
      └─ supabase_client.py
    └─ models/
      └─ recipe.py
      └─ search.py
    └─ utils/
      └─ config.py
  └─ data_pipeline/
    └─ download_datasets.py
    └─ process_data.py
    └─ add_vibes.py
    └─ generate_embeddings.py
    └─ supabase_loader.py
  └─ requirements.txt
  └─ .env
  └─ venv/
|
└─ docs/
  └─ PROJECT_SCOPE.md
  └─ IMPLEMENTATION_GUIDE.md (this file)
```

Backend Core Architecture

FastAPI Routes

python

```
# POST /recipes/by-vibe
# Input: {"vibe": "cozy comfort food"}
# Process: Embed → Vector search → Filter → RAG explain → Return recipes

# POST /recipes/by-inventory
# Input: {"ingredients": ["chicken", "tomato"]}
# Process: Match recipes with ingredients → Return ranked list

# POST /recipes/combined
# Input: {"vibe": "...", "ingredients": [...], "max_cook_time": 30}
# Process: Embed vibe → Search → Filter by constraints → Return top 5

# GET /recipes/{id}
# Input: Recipe ID
# Output: Full recipe details

# GET /recipes/{id}/explain
# Input: Recipe ID, user vibe
# Output: RAG-generated explanation

# GET /health
# Output: {"status": "ok", "services": {...}}
```

RAG Pipeline Flow

```
User Input
↓
1. EMBEDDING (Sentence-Transformers)
  ↳ Convert text to 384-dim vector
↓
2. VECTOR SEARCH (Supabase pgvector)
  ↳ SELECT * FROM recipes
    ORDER BY 1 - (vibe_embedding <=> query_vector)
    LIMIT 10
↓
3. FILTERING (SQL)
  ↳ Apply: ingredients, cook_time, cuisine constraints
↓
4. RANKING
  ↳ Sort by relevance score
↓
5. GENERATION (Ollama + Mistral)
  ↳ Create prompt with top 5 recipes
```

Send to LLM

Generate explanation



Response

↳ Recipes + Explanation + Metadata

Database Schema (Supabase)

sql

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
CREATE TABLE recipes (  
  id BIGSERIAL PRIMARY KEY,  
  name TEXT NOT NULL,  
  description TEXT,  
  ingredients JSONB,          -- [{"name": "...", "amount": "...", "unit": "..."}]  
  instructions JSONB,        -- ["step1", "step2", ...]  
  vibes JSONB,               -- ["cozy", "comfort", "warm"]  
  cuisine TEXT,  
  cook_time_minutes INTEGER,  
  difficulty TEXT,           -- easy, medium, hard  
  vibe_embedding vector(384), -- Main search vector  
  source TEXT,               -- food_com, recipe_nlg, indian_recipes  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE INDEX idx_vibe_embedding ON recipes USING ivfflat (vibe_embedding vector_cosine_ops);  
CREATE INDEX idx_cuisine ON recipes(cuisine);  
CREATE INDEX idx_cook_time ON recipes(cook_time_minutes);  
CREATE INDEX idx_vibes ON recipes USING GIN (vibes);
```

Data Pipeline (Execution Order)

Step 1: Download Datasets (Kaggle)

bash

```
pip install kaggle

# Download from Kaggle:
kaggle datasets download -d shuyangli94/food-com-recipes-and-user-interactions
kaggle datasets download -d paulthomothymooney/recipe_nlg
kaggle datasets download -d madhav28/indian-recipes-dataset
```

Output: 3 CSV/JSON files with recipes

Step 2: Process & Combine Data

```
python

# backend/data_pipeline/process_data.py

import pandas as pd
import json

# Load all datasets
food_com = pd.read_csv('recipes.csv')
recipe_nlg = json.load(open('full_recipe_2022_08_02.json'))
indian = pd.read_csv('Indian_Food_Dataset_With_Link_new.csv')

# Standardize to common format:
# {name, description, ingredients[], instructions[], cuisine, cook_time, difficulty,

# Combine into single list
# Save as combined_recipes.csv
```

Step 3: Add Vibe Tags

```
python
```

```
# backend/data_pipeline/add_vibes.py

vibe_keywords = {
    'cozy': ['comfort', 'warm', 'nostalgic', 'homemade'],
    'energetic': ['quick', 'healthy', 'protein', 'breakfast'],
    'date-night': ['elegant', 'romantic', 'fancy', 'rich'],
    'healthy': ['salad', 'vegetables', 'light', 'low-cal'],
    'adventurous': ['spicy', 'exotic', 'international', 'bold'],
    # ... 7 more vibes
}

# For each recipe: Find matching vibes from keywords
# Add to vibes column
# Save recipes_with_vibes.csv
```

Step 4: Generate Embeddings

```
python

# backend/data_pipeline/generate_embeddings.py

from sentence_transformers import SentenceTransformer

model = SentenceTransformer('BAAI/bge-small-en-v1.5')

# For each recipe:
text = f"{recipe['name']} {recipe['description']}"
embedding = model.encode(text) # Returns 384-dim vector

# Save embeddings (takes 4-6 hours for 2.5M recipes on CPU)
```

Step 5: Load to Supabase

```
python
```

```
# backend/data_pipeline/supabase_loader.py

from supabase import create_client

supabase = create_client(SUPABASE_URL, SUPABASE_KEY)

# Batch insert recipes with embeddings
# Create indexes
# Test vector search
```

Backend Implementation Details

main.py (FastAPI Setup)

```
python

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routes import recipes

app = FastAPI(title="Recipe Vibe API")

# CORS for frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["localhost:5173", "localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Routes
app.include_router(recipes.router)

@app.get("/health")
async def health():
    return {"status": "ok"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

rag_pipeline.py (Core RAG Logic)

python

```
from langchain.embeddings import SentenceTransformerEmbeddings
from langchain.vectorstores import SupabaseVectorStore
from langchain.llms import Ollama
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

embeddings = SentenceTransformerEmbeddings(model_name="BAAI/bge-small-en-v1.5")
vectorstore = SupabaseVectorStore(client=supabase, embedding=embeddings, table_name=
llm = Ollama(base_url="http://localhost:11434", model="mistral")

def search_recipes_by_vibe(vibe: str, max_results: int = 5):
    # 1. Retrieve top 10 recipes
    docs = vectorstore.similarity_search(vibe, k=10)

    # 2. Format for LLM
    recipes_text = "\n".join([d.page_content for d in docs])

    # 3. Create prompt
    prompt = PromptTemplate(
        input_variables=["vibe", "recipes"],
        template="""
        User is looking for: {vibe}

        Here are matching recipes:
        {recipes}

        Explain why each recipe matches their vibe (2-3 sentences each).
        """
    )

    # 4. Generate explanation
    chain = LLMChain(llm=llm, prompt=prompt)
    explanation = chain.run(vibe=vibe, recipes=recipes_text)

    return {
        "recipes": [doc.metadata for doc in docs[:max_results]],
        "explanation": explanation
    }
```


embeddings.py

python

```
from sentence_transformers import SentenceTransformer

class EmbeddingsService:
    def __init__(self):
        self.model = SentenceTransformer('BAAI/bge-small-en-v1.5')

    def embed(self, text: str):
        return self.model.encode(text, convert_to_numpy=True).tolist()
```

llm_service.py

python

```
from langchain.llms import Ollama

class LLMService:
    def __init__(self):
        self.llm = Ollama(
            base_url="http://localhost:11434",
            model="mistral",
            temperature=0.7
        )

    def generate(self, prompt: str):
        return self.llm(prompt)
```

supabase_client.py

python

```
from supabase import create_client
import os

class SupabaseClient:
    def __init__(self):
        self.client = create_client(
            os.getenv("SUPABASE_URL"),
            os.getenv("SUPABASE_ANON_KEY")
        )

    def search_recipes(self, embedding, limit=10):
        response = self.client.rpc(
            'match_recipes',
            {'query_embedding': embedding, 'match_count': limit}
        ).execute()
        return response.data

    def get_recipe(self, recipe_id):
        response = self.client.table('recipes').select('*').eq('id', recipe_id).single_get()
        return response.data
```

recipes.py (Routes)

python

```

from fastapi import APIRouter, HTTPException
from app.services.rag_pipeline import search_recipes_by_vibe
from app.services.embeddings import EmbeddingsService
from app.models.search import RecipeSearchRequest

router = APIRouter(prefix="/recipes", tags=["recipes"])
embedder = EmbeddingsService()

@router.post("/by-vibe")
async def recipes_by_vibe(request: RecipeSearchRequest):
    try:
        embedding = embedder.embed(request.vibe)
        results = search_recipes_by_vibe(request.vibe, request.max_results)
        return results
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/by-inventory")
async def recipes_by_inventory(ingredients: list[str]):
    # Supabase query: WHERE ingredients contains all items
    pass

@router.post("/combined")
async def recipes_combined(
    vibe: str,
    ingredients: list[str] = None,
    max_cook_time: int = None,
    cuisine: str = None
):
    # Embed vibe → Search → Filter by constraints → Return
    pass

@router.get("/{recipe_id}")
async def get_recipe(recipe_id: int):
    # Return full recipe details
    pass

@router.get("/{recipe_id}/explain")
async def explain_recipe(recipe_id: int, vibe: str):
    # RAG explain why this recipe matches vibe
    pass

```

Frontend Implementation Details

App.jsx (Main Component)

```
jsx

import { useState } from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import RecipeDetail from './pages/RecipeDetail';

export default function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/recipe/:id" element={<RecipeDetail />} />
      </Routes>
    </Router>
  );
}
```

pages/Home.jsx

```
jsx
```

```

import { useState } from 'react';
import { useRecipeSearch } from '../hooks/useRecipeSearch';
import MoodSelector from '../components/MoodSelector';
import InventoryInput from '../components/InventoryInput';
import RecipeList from '../components/RecipeList';
import LoadingState from '../components/LoadingState';

export default function Home() {
  const [vibe, setVibe] = useState('');
  const [ingredients, setIngredients] = useState([]);
  const { recipes, explanation, loading, search } = useRecipeSearch();

  const handleSearch = async () => {
    await search({
      vibe: vibe,
      ingredients: ingredients,
      max_results: 5
    });
  };

  return (
    <div className="min-h-screen bg-gradient-to-br from-orange-50 to-red-50">
      <div className="container mx-auto px-4 py-8">
        <h1 className="text-4xl font-bold text-gray-800 mb-8">
          🔍 What's Your Food Vibe?
        </h1>

        <div className="grid md:grid-cols-2 gap-8">
          /* Left Column: Inputs */
          <div className="space-y-6">
            <MoodSelector vibe={vibe} setVibe={setVibe} />
            <InventoryInput
              ingredients={ingredients}
              setIngredients={setIngredients}
            />
            <button
              onClick={handleSearch}
              disabled={loading || !vibe}
              className="btn btn-primary w-full"
            >
              {loading ? 'Searching...' : 'Find Recipes'}
            </button>
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

    { /* Right Column: Results */ }
    <div>
      {loading ? (
        <LoadingState />
      ) : recipes.length > 0 ? (
        <div>
          <p className="text-gray-700 mb-4">{explanation}</p>
          <RecipeList recipes={recipes} />
        </div>
      ) : (
        <p className="text-gray-500">Start by describing your vibe...</p>
      )}
    </div>
  </div>
</div>
</div>
);
}

```

components/MoodSelector.jsx

```
jsx
```

```

export default function MoodSelector({ vibe, setVibe }) {
  const quickVibes = ['Cozy', 'Energetic', 'Adventurous', 'Healthy', 'Indulgent'];

  return (
    <div className="card bg-white shadow-lg">
      <div className="card-body">
        <h2 className="card-title">Describe Your Vibe</h2>

        <textarea
          value={vibe}
          onChange={(e) => setVibe(e.target.value)}
          placeholder="e.g., cozy comfort food, quick weeknight dinner..."
          className="textarea textarea-bordered w-full h-24"
        />

        <div className="flex flex-wrap gap-2 mt-4">
          {quickVibes.map(v => (
            <button
              key={v}
              onClick={() => setVibe(v.toLowerCase())}
              className="btn btn-sm btn-outline"
            >
              {v}
            </button>
          ))}
        </div>
      </div>
    </div>
  );
}

```

components/RecipeCard.jsx

```

jsx

```

```

import { Link } from 'react-router-dom';

export default function RecipeCard({ recipe }) {
  return (
    <div className="card bg-white shadow-md hover:shadow-lg transition">
      <div className="card-body">
        <h3 className="card-title text-lg">{recipe.name}</h3>
        <p className="text-gray-600 text-sm">{recipe.description}</p>

        <div className="flex gap-2 flex-wrap mt-2">
          {recipe.vibes?.map(vibe => (
            <span key={vibe} className="badge badge-primary">
              {vibe}
            </span>
          ))}
        </div>

        <div className="flex justify-between text-sm text-gray-500 mt-4">
          <span>🕒 {recipe.cook_time_minutes} min</span>
          <span>📊 {recipe.difficulty}</span>
        </div>

        <div className="card-actions justify-end mt-4">
          <Link to={`/recipe/${recipe.id}`} className="btn btn-sm btn-primary">
            View Recipe
          </Link>
        </div>
      </div>
    </div>
  );
}

```

hooks/useRecipeSearch.js

```
javascript
```



```
import { useState } from 'react';
import api from '../services/api';

export function useRecipeSearch() {
  const [recipes, setRecipes] = useState([]);
  const [explanation, setExplanation] = useState('');
  const [loading, setLoading] = useState(false);

  const search = async (params) => {
    setLoading(true);
    try {
      const response = await api.post('/recipes/by-vibe', {
        vibe: params.vibe,
        ingredients: params.ingredients,
        max_results: params.max_results || 5
      });

      setRecipes(response.data.recipes);
      setExplanation(response.data.explanation);
    } catch (error) {
      console.error('Search failed:', error);
    } finally {
      setLoading(false);
    }
  };

  return { recipes, explanation, loading, search };
}
```

services/api.js

javascript

```
import axios from 'axios';

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000';

const api = axios.create({
  baseURL: API_URL,
  headers: {
    'Content-Type': 'application/json'
  }
});

export default api;
```

Setup Instructions (macOS)

1. Clone & Initialize Project

```
bash

mkdir recipe-vibe-app && cd recipe-vibe-app
git init
```

2. Set up Frontend

```
bash

npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install -D tailwindcss daisyui postcss autoprefixer axios react-router-dom
npm run dev
# Frontend runs on http://localhost:5173
```

3. Set up Backend

```
bash
```

```
cd ..
python3 -m venv venv
source venv/bin/activate

# Create requirements.txt:
# fastapi==0.104.1
# uvicorn==0.24.0
# langchain==0.1.0
# supabase==2.0.3
# sentence-transformers==2.2.2
# ollama==0.0.54
# python-dotenv==1.0.0

pip install -r requirements.txt
python backend/app/main.py
# Backend runs on http://localhost:8000
```

4. Set up Ollama

```
bash

# Download from ollama.ai
# Then:
ollama pull mistral
ollama serve
# Ollama runs on http://localhost:11434
```

5. Set up Supabase

```
bash

# Create account at supabase.com
# Create project → Get connection string
# Create .env in backend/:
SUPABASE_URL=your_url
SUPABASE_ANON_KEY=your_key
```

6. Run Data Pipeline

```
bash
```

```
cd backend/data_pipeline
```

```
# Step 1: Download datasets
```

```
python download_datasets.py
```

```
# Step 2: Process & combine
```

```
python process_data.py
```

```
# Step 3: Add vibes
```

```
python add_vibes.py
```

```
# Step 4: Generate embeddings (takes 4-6 hours)
```

```
python generate_embeddings.py
```

```
# Step 5: Load to Supabase
```

```
python supabase_loader.py
```

Vibe Categories (12 Core)

1. COZY - warm, nostalgic, homemade, comfort
2. ENERGETIC - quick, healthy, protein, energizing
3. DATE-NIGHT - elegant, romantic, impressive
4. HEALTHY - light, salad, vegetables, nutritious
5. ADVENTUROUS - spicy, exotic, international, bold
6. INDULGENT - decadent, creamy, rich, dessert
7. FAMILY-FRIENDLY - simple, kid-approved, classic
8. QUICK-WEEKNIGHT - easy, fast (<30 min), minimal prep
9. WEEKEND-SPECIAL - impressive, detailed, showstopper
10. SEASONAL - fresh, garden, harvest, berries
11. LIGHT & FRESH - Mediterranean, olive oil, herbs
12. COMFORT-INDULGENT - hearty, rich, satiating

Testing Locally

```
bash
```

```
# 1. Start all services
# Terminal 1: Frontend
cd frontend && npm run dev

# Terminal 2: Backend
cd backend && source venv/bin/activate && python main.py

# Terminal 3: Ollama
ollama serve

# Terminal 4: Test API
curl -X POST http://localhost:8000/recipes/by-vibe \
  -H "Content-Type: application/json" \
  -d '{"vibe": "cozy comfort food"}'
```

Environment Variables (.env)

```
# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your-anon-key

# Ollama
OLLAMA_BASE_URL=http://localhost:11434

# FastAPI
FASTAPI_ENV=development
LOG_LEVEL=INFO

# Frontend (.env in frontend/)
VITE_API_URL=http://localhost:8000
```

Common Errors & Fixes

Error	Fix
Ollama not responding	Ensure <code>ollama serve</code> is running in separate terminal
Supabase connection failed	Check SUPABASE_URL and SUPABASE_ANON_KEY in .env

Error	Fix
Frontend can't reach backend	Check CORS middleware in FastAPI, ensure backend on :8000
Slow embeddings	Use GPU (CUDA/Metal) or accept 4-6 hour runtime
Port already in use	Change port or kill process: <code>lsof -i :8000</code>

Key Files to Create

Backend:

- `app/main.py` - FastAPI setup (100 lines)
- `app/routes/recipes.py` - All endpoints (200 lines)
- `app/services/rag_pipeline.py` - RAG logic (300 lines)
- `app/services/embeddings.py` - Embeddings wrapper (50 lines)
- `app/services/llm_service.py` - Ollama integration (50 lines)
- `app/services/supabase_client.py` - DB operations (100 lines)
- `data_pipeline/process_data.py` - Data cleaning (300 lines)
- `data_pipeline/add_vibes.py` - Vibe tagging (150 lines)
- `data_pipeline/generate_embeddings.py` - Embedding generation (200 lines)
- `data_pipeline/supabase_loader.py` - Database loading (150 lines)

Frontend:

- `components/MoodSelector.jsx` (100 lines)
- `components/InventoryInput.jsx` (150 lines)
- `components/RecipeCard.jsx` (120 lines)
- `components/RecipeList.jsx` (100 lines)
- `components/LoadingState.jsx` (50 lines)
- `pages/Home.jsx` (200 lines)
- `pages/RecipeDetail.jsx` (150 lines)
- `hooks/useRecipeSearch.js` (100 lines)
- `services/api.js` (50 lines)
- `App.jsx` (50 lines)

Total: ~2,800 lines of code

Deployment

Frontend → Vercel

```
bash  
  
cd frontend  
npm run build  
# Connect GitHub → Auto-deploys
```

Backend → Railway

```
1. Connect GitHub repo to Railway  
2. Add env vars (SUPABASE_URL, SUPABASE_KEY)  
3. Auto-detects Python → Deploys  
4. Disable Ollama locally, use API instead (future)
```

Next Steps

1. ☒ Read PROJECT_SCOPE.md for full understanding
 2. ☒ Follow Setup Instructions
 3. ☒ Download datasets from Kaggle
 4. ☒ Run data pipeline (takes ~8 hours total)
 5. ☒ Create backend files
 6. ☒ Create frontend files
 7. ☒ Test locally
 8. ☒ Deploy to Vercel + Railway
-

Version: 1.0

For: Claude Opus 4.7

Status: Implementation Reference